Active Imitation Learning for a Neural Network That Drives a Car in CARLA

DRAFT v0.6. INCOMPLETE MANUSCRIPT. NOT FOR CIRCULATION.

Medhansh Khattar, Prashant Kumar Jaglan, Yash Yadav
Samyak Jain, Sanchit Kumar Sinha, Aryan Garg
Vellore Institute of Technology, Bhopal, India

***Abstract*—We describe an imitation learning pipeline for a neural network that drives a car in CARLA 0.9.16. The policy is a small multilayer perceptron of 667 parameters that reads eight scalars describing the car relative to its route and outputs steering, accelerator and brake. Three contracts hold the system together: an observation contract of eight fields, an action protocol of seven fields, and a deployability contract that an exported model must satisfy before the runtime will let it drive. A safety supervisor runs outside the policy on the simulator host and keeps full braking authority.**

The title names our aim, not our current state: active imitation, where the learner drives and the expert corrects the states it visits, is the next milestone, and what runs today is passive cloning. The method is complete, but the evaluation is not. We cannot yet say how well any of it drives, because we cannot keep the simulator rendering long enough to record one complete route. Camera compositing saturates the integrated GPU at speed, environment streaming stalls the client, and the crashes that follow end episodes partway through. Our acceptance rule discards any interrupted route, so these crashes show up as an empty training set: zero accepted routes against a target of twelve. *Sections VII to IX are left unwritten.*

***Index Terms*—autonomous driving, active imitation learning, behaviour cloning, CARLA, rendering bottleneck, safety supervision, reproducibility.**

I. Introduction

CARLA [1] is the most widely used open simulator for driving research. It runs an Unreal Engine world behind an RPC interface, supplies OpenDRIVE road topology, traffic and weather, and replays a seeded route deterministically after a code change. Crashes cost nothing, which is the usual argument for working in simulation. That argument assumes the simulator keeps running.

Most work since Bojarski *et al.* [2] maps camera pixels straight to control. We started somewhere narrower. Our policy reads eight scalars measured relative to the route instead of images, mainly because of how few demonstrations six undergraduates can curate: on a few thousand frames, a larger network learns the shape of the training town rather than how to stay in a lane [4]. What follows is the architecture, the contracts holding it together, and the rendering failures we spent most of the project fighting. Where a driving result would normally go, we say so, and Section VI explains why.

II. Related Work

Behaviour cloning for driving was shown at scale by Bojarski *et al.* [2]. Codevilla *et al.* added a command input so one network could resolve what to do at an intersection [3], and later catalogued what cloning gets wrong on its own [4]: distributional shift, and a bias toward whatever action the expert takes most often. Ross *et al.* give the usual fix, DAgger [5], in which the expert labels the states the learner visits. That is the active imitation of our title, and the step we have not reached. Within CARLA, Learning by Cheating [6] beats direct sensorimotor cloning by distilling from a privileged agent, and TransFuser [7] fuses several sensors. Our observation contract sits closer to the privileged stage of [6].

> *TODO (Related Work). Still to read and cite: Toromanoff et al. on implicit affordances (CVPR 2020); the Kiran et al. DRL survey (2021), for positioning; SAE J3016, so the autonomy claim is precise; and anything on simulator throughput, which we have not looked at and which bears on Section VI.*

III. System Architecture

Two machines share the work over a direct Ethernet link. Machine 1 runs the CARLA server, which owns physics, the OpenDRIVE map, the traffic manager, sensors and the vehicle. Machine 2 runs inference, the route planner, an observer window and network visualisation. Each iteration goes: machine 1 publishes world state, machine 2 runs the planner and the policy, machine 2 sends back a driving command, and machine 1's safety layer checks that command before applying it.

We split the map in two. The physical map stays with the simulator, which owns the world. The navigation map needs only road topology to pick a destination and lay out waypoints, so it can live on machine 2. A watchdog on machine 1 is meant to brake if the link drops. It is written, but we have not tested it by pulling the cable.

IV. Methodology

A. Observation contract

`policy_contract.py` defines what the network sees: the eight scalars in Table I, each clamped by a normaliser the same module owns.

Working in coordinates relative to the route rather than to

TABLE I
THE OBSERVATION CONTRACT, EIGHT FIELDS

Field	Meaning	Field	Meaning
`lane_error_m`	offset from lane centre	`front_obst_m`	nearest car ahead
`heading_err_deg`	car versus road angle	`red_light`	binary: light governs
`speed_kmh`	current road speed	`curvature`	bend ≈8 m ahead
`target_spd_kmh`	speed controller wants	`stopped`	binary: <1 km/h

the world is what lets a policy trained on one town be evaluated on another. One gap is still open. The waypoint used as the route reference when computing heading error and curvature during training is not handed to the policy the same way at inference, so the two can disagree on those two fields. Any number we report before closing this would be suspect.

B. Action protocol and safety supervision

The policy produces three values and the vehicle interface takes seven. Steering comes out in $[-1, +1]$ and maps to $\pm 70°$. Accelerator and brake are clamped to $[0, 1]$, and the runtime will not apply both at once. The output layer is linear, so all clamping happens in the runtime rather than inside the network. Clutch, handbrake, wipers and gear come from a deterministic transmission model with six gears. CARLA does not simulate engine speed, so we estimate it from road speed and gear ratio and label it *estimated telemetry* everywhere it appears.

The safety supervisor runs on the simulator host, outside the policy. It forces a full brake on collision, sensor timeout, a command older than 0.5 s, lane error above 2 m, an obstacle inside 7 m, speed above 50 km/h, a red light, or telemetry that fails validation. Nothing else is allowed to override the network.

C. Recording demonstrations

We record from two experts: CARLA's `BasicAgent`, which follows fixed rules, and a person driving the cockpit. A route runs from a spawn point to a destination at least 150 m away, and each tick appends one JSONL row with the eight observations, the expert's three controls, telemetry and scenario metadata. A route is kept only if it finished, hit nothing, and holds at least 100 samples. Crashed and interrupted runs are thrown out rather than trimmed. That rule matters for Section VI, because it is what turns rendering failures into missing data instead of bad data.

Recording happens on Town05. Town04 is held back as an unseen map, so a good final score cannot be explained by the network having memorised the road. Twelve Town04 teacher episodes recorded before we adopted this rule are quarantined, not deleted.

D. Training

Training runs offline in a Colab notebook and writes a JSON artifact the local driver loads. The network is a linear layer from 8 to 24, tanh, a linear layer from 24 to 16, tanh, then a linear head from 16 to 3, giving 667 parameters. We use AdamW at $\mathrm{lr} = 10^{-3}$ with weight decay 10^{-5}, mean squared error over the three control channels, batches of 128, at most 80 epochs, and early stopping after 12 epochs without improvement. Seed 7 is set for Python, NumPy and PyTorch, and everything runs on CPU.

Splits are taken over whole routes, 70/15/15, never over single frames. Neighbouring frames are nearly identical, so splitting at random puts copies of the same moment in both train and test and the reported error stops meaning anything. Each episode lands in one split, and an assertion checks the three are disjoint. The notebook writes its normaliser straight out of the contract file, so training and inference cannot drift apart.

E. Deployability gate

Every exported artifact carries a contract version, a `deployable` flag and the episode names in each split. The driver rejects anything that is not contract v2, or that has the flag set false. The only artifact we have at the moment is a synthetic smoke test at contract v1 with `deployable` false, so it gets rejected and route mode stays unavailable. There is no scripted fallback, so the car simply does not drive itself. Training is disabled in the runtime, which means a deployed model cannot quietly retrain.

One runtime behaviour is not learned, and we should declare it. Cloning produces too much braking, so the policy tends to sit still on an empty road. A bounded recovery capped at 0.35 accelerator releases the stall, switches off at once for a red light, an obstacle or bad telemetry, and never touches the weights. Any result has to say whether it was on.

V. EXPERIMENTAL SETUP (PARTIAL)

The scenario catalogue names four categories, so an experiment can ask for a category instead of fixing a map in code: urban (Town03, Town04, Town05, Town10HD, 30 NPCs), highway (Town05, Town06, Town07, 20), rural (Town01, Town02, 12) and intersection (Town03, Town04, 40). Weather comes from eight presets and road damage from three pothole modes. All of this is configuration. No category has completed a verified evaluation run, for the reasons in the next section.

> *TODO (Section V). Missing: a hardware table for both machines; control loop latency over the link (mean, p99); headless tick rate; the commit hash for the reported runs.*

VI. THE RENDERING BOTTLENECK

Most of our difficulty has come from rendering rather than from the learning algorithm. We cannot reliably keep the environment and the camera image rendering long enough to finish a single route.

A. Rendering the camera image at speed

The visual cockpit composites four camera feeds on top of streamed scenery. On an Intel Arc 140V integrated GPU that is too much. Frame rate falls as road speed rises, because

travelling faster makes Unreal stream in new scenery while the same GPU is still compositing the camera panel. It does not degrade gently. The image stalls, the client falls behind the server tick, and the commands reaching the car go stale.

We have two mitigations, and both cost realism. The default neural path runs *headless*, with no camera feeds at all. On the visual build, throttle is cut above 42 km/h and top speed capped at 48 km/h to keep the GPU alive. The speed range we can demonstrate in is therefore set by a rendering limit rather than by anything about driving, and we cannot record behaviour at highway speed at all.

B. Environment rendering and process failures

Environment rendering fails in three ways, all ending the episode:

- **Thermal termination.** The GPU shares a thermal package with the CPU. One afternoon of sustained rendering produced both a fatal hardware error and a critical thermal hibernation. This hardware exposes no live temperature counter, so our health guard reads the Windows Event Log instead and refuses to launch during a cooldown of 30 or 60 minutes.
- **Silent process death.** A rendering component wedges, Windows kills it as a hung application, and the rest of the stack follows. For a long time nothing was logged, because the old launcher fired each process into a detached task whose output went nowhere. One supervisor now owns both processes, writes logs, checks liveness by probing the RPC port rather than trusting a PID, and restarts with a backoff from 5 s to 120 s.
- **Orphaned renderers.** `CarlaUE4.exe` is a shim around another executable, and the virtualenv `python.exe` starts the real interpreter again as a child. Killing the visible process left the real renderer holding port 2000, so the next launch fought the previous one. The supervisor now kills process trees and sweeps orphans before starting.

C. How this empties the training set

These failures are why Section VII is empty. A rendering stall interrupts a route midway, and by the acceptance rule above an interrupted route is discarded rather than kept, which we chose on purpose: a route ending in a crash teaches the policy that crashing is ordinary. The two rules do not sit well together. The more often the environment fails to render, the fewer routes survive, and we currently have zero accepted routes out of twelve. The network is not failing to learn. We have not given it anything to learn from.

> *TODO (Section VI). Qualitative; make quantitative before submission. Needed: (a) FPS against road speed, headless against visual, as a figure; (b) mean route survival time before a rendering termination; (c) counts per failure class; (d) package temperature at termination (unobtainable, so state it); (e) does splitting across two machines help.*

VII. Results

This section is not yet written.

There is no corpus to write it from, for the reasons above, and a training loss against the smoke test artifact would say nothing. To stop ourselves picking metrics after seeing results, we are fixing the format now: collisions per 100 km, lane departures per km, route completion rate, safety interventions per route, and mean absolute error for each control channel, each for the scripted baseline and for the learned policy on the map we held back.

> *TODO (Section VII). Prerequisites, in order: (1) stabilise rendering so a route of 150 m survives from start to finish; (2) accept at least 12 Town05 routes; (3) close the training and inference gap in Section IV; (4) export a contract v2 artifact with* `deployable` *set true; (5) run the Town04 evaluation with the stall assist on and off. No prose before (5) completes.*

VIII. Discussion

Stub. Only the observations that do not depend on results.

Most of the time on this project went to rendering and process management rather than to modelling, which is not something the papers we started from discuss. Results in [6] and [7] assume a simulator that renders reliably. When it does not, the work moves somewhere those papers have no reason to go.

> *TODO (Section VIII). Once VII exists: (a) a leakage demo, error under splitting by route against by frame; (b) a failure taxonomy from Town04; (c) what eight scalars cannot represent (pedestrians, signs, anything needing vision); (d) does headless collection bias the policy.*

IX. Conclusion and Future Work

[Not written. To be drafted after Sections VI and VII are complete.]

Planned next steps, in rough order. Get the rendering stable, moving the renderer to dedicated hardware if that is what it takes. Replace the eight scalars with RGB, depth and semantic segmentation. Add the command input of [3] and the active correction loop of [5], which is what would make our title accurate. Train across the whole catalogue rather than one town. Reward logging already runs every tick, but nothing consumes it.

References

[1] A. Dosovitskiy *et al.*, "CARLA: An Open Urban Driving Simulator," *CoRL*, 2017. arXiv:1711.03938.

[2] M. Bojarski *et al.*, "End to End Learning for Self-Driving Cars," NVIDIA, 2016. arXiv:1604.07316.

[3] F. Codevilla *et al.*, "End-to-end Driving via Conditional Imitation Learning," *ICRA*, 2018. arXiv:1710.02410.

[4] F. Codevilla *et al.*, "Exploring the Limitations of Behavior Cloning for Autonomous Driving," *ICCV*, 2019.

[5] S. Ross, G. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," *AISTATS*, 2011. arXiv:1011.0686.

[6] D. Chen *et al.*, "Learning by Cheating," *CoRL*, 2019. arXiv:1912.12294.

[7] A. Prakash, K. Chitta, and A. Geiger, "Multi-Modal Fusion Transformer for End-to-End Autonomous Driving," *CVPR*, 2021.

TODO (References). Incomplete: add CARLA docs, OpenDRIVE, rendering refs.